Replikationsarbeit zu "Artificial hivemind: The open-ended homogeneity of language models (and beyond)"

Marko Max Lämmerhirt, Marcel Haase, Tuan Duy Nguyen

13. April 2026

1 Einleitung

Die rasante Verbreitung und die starke Nachfrage nach großen Sprachmodellen (engl. Large Language Model – LLM) machen diese zu einer zentralen Technologie im Kern moderner KI-Systeme.[1] Das breite Einsatzspektrum von LLMs bietet Möglichkeiten für die Anwendung in Suchmaschinen[2], Programmierassistenten[3] oder als Werkzeuge für die Bild- und Textgestaltung[4]. Bei der Verwendung von LLMs für kreative Aufgaben, insbesondere bei der Beantwortung offener Fragen, zeigen sich häufig Defizite hinsichtlich der Reproduktion menschenähnlicher Kreativität. Durch die weite Verbreitung und starke Nutzung von LLMs trägt der generierte Output zur Formung des Gedankenguts der Nutzer bei. Wenn verschiedene Modelle jedoch keinen kreativen oder differenzierten Output generieren können, kommt es langfristig zu einer Homogenisierung.[5]

Der replizierte Forschungsartikel Artificial Hivemind: The Open-Ended Homogeneity of Language Models (and Beyond) betrachtet das Thema der Homogenität von LLM-Outputs genauer. Die Arbeit von Jiang et al. (2025) nutzt einen Datensatz aus offenen Fragen, um die Homogenitäten der Sprachmodelle bei der Beantwortung der Fragen zu untersuchen. Jiang et al. (2025) konnte aufzeigen, dass sowohl Intra- als auch Intermodell-Beziehungen eine starke Homogenität in den generierten Outputs aufweisen.

Die folgende Arbeit stellt eine Replikation des Forschungsartikels Jiang et al. (2025) bezüglich der Untersuchungen der Intra- und Intermodellhomogenitäten dar. In Anknüpfung daran stellen wir einen Ansatz zur Verringerung der Intra- sowie Intermodellhomogenitäten vor.

2 Umfang der Replikation/Reproduktion

Die Originalarbeit von Jiang et al. (2025) stützt die Untersuchung auf drei Forschungsfragen:

(FF1) Welche Typen offener Fragen stellen Nutzer?

[1] L. Jiang et al. 2025, S. 1.
[2] Y. Gao et al. 2023, S. 16.
[3] Phung et al. 2023, S. 14.
[4] Sordo et al. 2025, S. 13.
[5] L. Jiang et al. 2025, S. 1.

Zur Beantwortung der FF1 stellt die Originalarbeit eine Taxonomie vor, die realitätsnahe und offene Fragen in sechs Über- und 17 Unterkategorien klassifiziert. Durch die Analyse der FF1 konnten die Autoren den INFINITY-CHAT100[6] Datensatz aus 100 offenen Fragen erstellen, welcher als Grundlage für die Untersuchung der Intra- und Intermodellhomogenitäten dient.[7]

> (FF2) Welche Intra- und Intermodellhomogenitäten weisen LLMs bei der Beantwortung offener Fragen auf?

Für die Untersuchung der FF2 werden für jede Frage des Datensatzes INFINITY-CHAT100 50 Samples pro Frage von jedem gewählten Modell generiert. Die gewonnenen Outputs werden innerhalb eines Modells sowie zwischen den Modellen mithilfe der Kosinus-Ähnlichkeit verglichen. Die Intramodellhomogenität kennzeichnet hierbei die Ähnlichkeiten von Outputs innerhalb eines Sprachmodells. Widerum bezeichnet die Ähnlichkeit der Outputs verschiedener Modelle die Intermodellhomogenität. Die Untersuchungen des Artikels zeigen in beiden Fällen eine hohe Ähnlichkeit in den generierten Antworten, sodass keine echte Antwortdiversität innerhalb eines Modells sowie zwischen unterschiedlichen Modellen bei sich überschneidenden Trainingseigenschaften oder weiteren Charakteristiken zu erwarten ist.[8]

> (FF3) Wie bewerten LMs, Reward Models und LM Judges alternative Antworten auf offene Fragen und wie weichen diese von menschlichen Bewertungen ab?

Zur Beantwortung der FF3 werden menschliche Bewertungen mit LM-Bewertungen verglichen. Ein Auszug der beantworteten Fragen aus FF2 wird sowohl von Menschen als auch durch LMs, Reward Models und LM-Judges bewertet. Die Ergebnisse dieses Vergleichs zeigen, dass bei starker menschlicher Uneinigkeit oder bei ähnlich guten Antworten die Ähnlichkeit der LM-Bewertungen, mit den menschlichen Wertungen, erheblich geringer ist.

Diese Replikationsarbeit konzentriert sich auf die Überprüfung der Ergebnisse der FF2. Dabei sollen für ausgewählte Modelle Antworten auf die offenen Fragen des Datensatzes INFINITY-CHAT100 generiert werden. Für jedes ausgewählte Modell sollen 50 Antwortsamples pro Frage generiert werden. Folgend soll die Intra- sowie die Intermodellhomogenitäten berechnet und als Heatmaps visualisiert werden. Anschließend wird ein Vergleich der Replikationsergebnisse mit den Originalergebnissen vorgenommen.

Darüber hinaus soll diese Arbeit einen Ansatz zur Verringerung der Intra- und Intermodellhomogenitäten untersuchen, indem die Fragen durch Systemprompting ergänzt werden. Unser Vorgehen schließt hierbei an die Methodik der Replikation an.

3 Methoden

3.1 Modellbeschreibung

Der replizierte Artikel nutzt für die Untersuchung der Intramodell- und Intermodellhomogenitäten ein Repertoire aus 79 Sprachmodellen.[9] Die LLMs reichen hierbei von kleinen 0,5 Milliarden Parameter Modellen, wie Qwen2.5-0.5B-Instruct, bis hin zu sehr großen Modellen, wie claude-3-5-sonnet-20241022 oder DeepSeek-V3 mit 685B Parametern. Für die Replikationsarbeit

[6] `https://huggingface.co/datasets/liweijiang/infinite-chats-eval`
[7] L. Jiang et al. 2025, S. 2–4.
[8] L. Jiang et al. 2025, S. 2, 4–5.
[9] L. Jiang et al. 2025, S. 31–32.

wurde eine Stichprobe über die Open-Source Modellauswahl der Originalarbeit gezogen, welche alle genutzten Modellfamilien berücksichtigt sowie mit unseren Ressourcenrestriktionen vereinbar war. Die konkreten Modelle dieser Replikationsarbeit sind in Tabelle 1 dargestellt.

Tabelle 1: Übersicht der ausgewählten Sprachmodelle

Modellname	**Parametergröße**
allenai/Llama-3.1-Tulu-3-8B	8B
meta-llama/Llama-3.1-8B-Instruct	8B
meta-llama/Llama-3.2-3B-Instruct	3B
mistralai/Ministral-8B-Instruct-2410	8B
mistralai/Mistral-7B-Instruct-v0.1	7B
mistralai/Mistral-Nemo-Instruct-2407	12B
Qwen/Qwen1.5-14B-Chat	14B
Qwen/Qwen2.5-14B-Instruct	14B
CohereForAI/aya-expanse-8b	8B
google/gemma-1.1-7b-it	7B
google/gemma-2-9b-it	9B
microsoft/phi-4	14B

Eine genaue Gegenüberstellung der von der Originalstudie verwendeten Modelle mit denen dieser Replikationsarbeit ist in Tabelle 2 zu finden.

Da sich unsere Replikation auf die Inferenz mit Sprachmodellen beschränkt, stand ein größerer Videospeicher im Vordergrund. Für die Berechnung kam ein NVIDIA DGX Spark mit 128 GB Unified Memory zum Einsatz. Für die Modellinferenz lässt sich der ungefähre Speicherbedarf eines Modells anhand seiner Parameter abschätzen. Bei der Inferenz mit 16-Bit (BF16) Parametern ergibt sich der Speicherbedarf als Anzahl der Parameter * 2 Byte. Bei einem 14B-Modell bedeutet dies einen ungefähren Speicherbedarf von 28 GB für die Parameter. Ebenfalls wurde die Sample-Generierung parallelisiert (10 Samples mit einem Prompt), wodurch Geschwindigkeitsgewinne erlangt wurden. Dadurch war jedoch zusätzlicher Speicherbedarf notwendig.

Die Auswahl großer Modelle mit anschließender Quantisierung haben wir bewusst ausgelassen, um eine methodische Abweichung von der Originalstudie zu vermeiden.

3.2 Datenbeschreibung

Als Datengrundlage wird der Datensatz INFINITY-CHAT100 verwendet. Dieser Datensatz besteht aus 100 offenen Fragen und stellt ein von Jiang et al. (2025) erstellten Auszug aus dem Datensatz INFINITY-CHAT[10] dar. Der Datensatz INFINITY-CHAT umfasst vielfältige, natürliche und offene Fragestellungen aus der realen Welt und wurde von Jiang et al. (2025) aus dem Datensatz WildChat[11] erhoben.[12] Auf diese offenen Fragen sind keine eindeutig korrekten Antworten möglich, wodurch ein breites Spektrum an Antwortmöglichkeiten erlaubt wird. Dieser Datensatz ermöglicht die Untersuchung der Antwortkompetenz von LLMs hinsichtlich der Generierung kreativer Antworten.

[10] `https://huggingface.co/datasets/liweijiang/infinite-chats-taxonomy`
[11] Zhao et al. 2024.
[12] L. Jiang et al. 2025, S. 2.

3.3 Hyperparameter

Die Werte der Hyperparameter wurden anhand der Festlegungen von Jiang et al. (2025) definiert. Dementsprechend wurde die Temperature auf 1,0 und Top-p auf 0,9 gesetzt. Die maximale Antwortlänge wurde auf 2048 Tokens und die Anzahl der Samples pro Frage auf 50 determiniert.[13]

3.4 Implementierung

Die Umsetzung der Replikation wurde in folgenden drei Schritten implementiert:

I. Generierung der Antworten

In diesem Schritt sollen die Antworten auf die offenen Fragen des INFINITY-CHAT100 Datensatzes durch die in Abschnitt 3.1 gewählten Modelle generiert werden. Die Originalarbeit von Jiang et al. (2025) stellt keinen spezifischen Code zur Generierung der Antworten durch die Modelle zur Verfügung. Daher erfolgt die Implementierung durch eigenen Code. Der zugehörige Code ist als Jupyter Notebook in der Programmiersprache Python 3.12.3 implementiert.[14] Zur Generierung der Antworten auf die offenen Fragen des Datensatzes wird in der Implementierung zuerst der Datensatz INFINITY-CHAT100 aus dem HuggingFace-Verzeichnis *liweijiang/infinite-chats-eval* geladen. Dabei wird die Library *datasets* von HuggingFace verwendet.

Folgend wird über jede Modell-ID der gewählten Modelle iteriert. In jeder Iteration werden folgende Operationen durchgeführt:

1. Der Tokenizer des jeweiligen Modells wird geladen.
2. Die Pipeline zur Generierung der Antworten wird für das jeweilige Modell gebaut.
3. Es werden mittels der Transformers-Klasse *GenerationConfig* die Hyperparameter sowie weitere Generierungsparameter gesetzt. Zur Beschleunigung der Sample-Generierung wird der Parameter *num_return_sequences* auf den Wert zehn gesetzt. Dadurch werden jeweils zehn unabhängige Samples auf eine Frage generiert. Der Wert wurde aus vorherigen Testversuche ermittelt, um die Ressourcenauslastung zu maximieren.
4. Die Anfrage wird in das passende Prompt-Format des jeweiligen Modells mittels der Methode *apply_chat_template* der Transformers-Klasse *AutoTokenizer* transformiert.
5. Über die vorher definierte Pipeline, werden für jede Frage im Datensatz 50 Samples generiert und in einer Textdatei gespeichert. Die 50 Samples pro Frage sind auf fünf Textdateien verteilt, die jeweils zehn Samples pro Frage beinhalten.

Der Schritt I speichert alle generierten Samples pro Frage im Pfad „Antworten/“ des Projekt-Verzeichnisses.

II. Generierung der Embeddings

In diesem Schritt werden die Embeddings für jedes generierte Sample ermittelt. Die Originalarbeit von Jiang et al. (2025) verwendet dafür das OpenAI Embedding-Modell *text-embedding-3-small*[15]. In diesem Schritt wird ebenfalls das Embedding-Modell *text-embedding-3-small* über die Library *openai* mit einem API-Token genutzt. Der Code wurde dafür

[13] L. Jiang et al. 2025, S. 2.
[14] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Generate/generate.ipynb`
[15] L. Jiang et al. 2025, S. 29.

in der Programmiersprache Python selbst erstellt. Die Implementierung aus der Originalarbeit ist zu diesem Schritt nicht veröffentlicht. Die Library *dotenv* wird verwendet, um die Umgebungsvariable für den OpenAI API-Token zu laden.

Für jedes Sample einer Frage jedes Modells wird eine API-Anfrage gesendet. Äquivalent wird zu jeder Datei im Verzeichnis „Antworten/“ mit der Endung „Teil_*.txt“ eine Datei mit den entsprechenden Embeddings für jedes Sample im Verzeichnis „Embeddings/“ im JSON-Format gespeichert. Die fünf Embedding-Dateien jedes Modells werden zusammengeführt, sodass eine Gesamtdatei entsteht. Diese enthält alle 50 Samples pro Frage für ein Modell.

III. Berechnung der Intra- und Intermodellhomogenitäten

Zur Berechnung der Intra- beziehungsweise der Intermodellhomogenität wird die Kosinus-Ähnlichkeit verwendet. Der Code der Originalarbeit[16] diente als Grundgerüst für die Berechnung der Kosinus-Ähnlichkeiten und für die grafische Darstellung der Ergebnisse als Heatmap.

1. Intramodellhomogenität

   Der Original-Code[17] wurde gekürzt und an das Format der gespeicherten Embeddings angepasst, da anhand der Originalarbeit nicht nachvollzogen werden konnte in welchem Format die Embeddings gespeichert wurden. Die Berechnung der Intramodellhomogenität erfolgt im Code[18] in folgenden Schritten:

   1. Die Embeddings zu jeder Antwort werden aus der jeweiligen JSON-Datei eines Modells geladen.
   2. Anschließend wird die paarweise Kosinus-Ähnlichkeit zwischen allen Paarkombinationen der Samples einer Frage berechnet und der Durchschnitt ermittelt. Diese Berechnung erfolgt für alle 100 Fragen mit jeweils 50 Samples. Das Ergebnis ist eine Liste der Länge 100, die für jede Frage die durchschnittliche Kosinus-Ähnlichkeit als Wert beinhaltet.
   3. Erhaltene Werte werden zur grafischen Darstellung in eine der zehn gleichgroßen Intervalle des Wertebereichs von 0-1 einsortiert. Danach wird die absolute Häufigkeit pro Intervall in Prozent umgerechnet.

2. Intermodellhomogenität

   Der Original-Code[19] wurde gekürzt und ebenfalls an das Format der gespeicherten Embeddings angepasst. Die Berechnung der Intermodellhomogenität folgt im Code[20] folgenden Schritten:

   1. Die Embeddings zu jeder Antwort werden aus der jeweiligen JSON-Datei eines Modells geladen.
   2. Für jedes Modellpaar und jede Frage wird für jede Sample-Kombination zwischen den Modellen die Kosinus-Ähnlichkeit berechnet. Folgend wird der Mittelwert über alle Kosinus-Ähnlichkeiten aller Paarkombinationen berechnet.

[16] `https://github.com/liweijiang/artificial-hivemind/tree/main/src/model_calibration_analysis`
[17] `https://github.com/liweijiang/artificial-hivemind/blob/main/src/model_calibration_analysis/main_analysis_within_model.py`
[18] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Similarity/intraSimilarity.ipynb`
[19] `https://github.com/liweijiang/artificial-hivemind/blob/main/src/model_calibration_analysis/main_analysis_across_models.py`
[20] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Similarity/interSimilarity.ipynb`

3. Die somit erhaltenen Mittelwerte werden zur grafischen Darstellung in einer Heatmap dargestellt. In der Heatmap werden alle Modellkombinationen mit ihren gemittelten Kosinus-Ähnlichkeiten abgebildet.

3.5 Aufbau der Experimente

Die Experimente wurden mithilfe der in Abschnitt 3.4 implementierten schrittweisen Logik durchgeführt. Insgesamt gab es zwei experimentelle Durchläufe dieser Pipeline. Der erste experimentelle Durchlauf hat die Replikation der Originalarbeit als Zielstellung. Der zweite experimentelle Durchlauf stellt eine Erweiterung dieser Replikationsarbeit, um einen möglichen Lösungsansatz zur Verringerung der Intra- sowie Intermodellhomogenität durch Prompt-Engineering dar.

Der erste experimentelle Durchlauf der Ergebnisreplikation ist wie folgt aufgebaut:
Im Schritt I (Generierung der Antworten) wurden, wie in Abschnitt 3.1 näher beschrieben, äquivalent zur Originalarbeit, ausgewählte LLMs zur Antwortgenerierung verwendet. Analog zur Originalarbeit wurde die gleiche Fragenbasis (Abschnitt 3.2), Hyperparameter sowie weitere Generierungsparameter (Abschnitt 3.3) gewählt. Zur Übergabe der Fragen an die Modelle wird ein Chat-Template genutzt. Dem Modell werden die zu beantwortenden Fragen in der Rolle *user* übergeben. Dieser Schritt wurde mit der Jupyter Notebook-Datei *generate.ipynb*[21] auf dem DGX Spark mit 128GB Unified Memory durchgeführt.

Der Schritt II (Generierung der Embeddings) wurde über die OpenAI-Plattform[22] mit einem API-Key durchgeführt. Das Jupyter Notebook-Datei *embeddings.ipynb*[23] wurde dabei zur Kommunikation über die API verwendet.

Der Schritt III (Berechnung der Homogenitäten) wurde mit dem angepassten Originalcode umgesetzt. Dabei wurden die Jupyter Notebook-Dateien *interSimilarity.ipynb*[24] und *intraSimilarity.ipynb*[25] verwendet.

Abweichend zum ersten experimentellen Durchlauf wird im zweiten experimentellen Durchlauf ein angepasster Prompt verwendet. Im Code[26] fungiert die Variable *with_system* als Schalter zwischen den Prompt-Arten der beiden experimentellen Durchläufe. Im zweiten experimentellen Durchlauf wird das ursprüngliche Chat-Template aus dem ersten Durchlauf um einen Systemprompt ergänzt. Dieser Prompt wird in der Rolle *system* übergeben und lautet „Answer the questions with a high degree of ingenuity and creativity, and don't hesitate to draw on unconventional ideas.“. Dieser Systemprompt wird jedes Mal mit der zu beantwortenden Frage übergeben und soll prüfen, ob diese Art des Promptings die Intra- und Intermodellhomogenität beeinflusst. Systemprompts sind meist vorgegeben und können nicht immer vom Nutzer beeinflusst werden. Der Inhalt des Systemprompt nimmt hierbei eine übergeordnete Rolle ein, welche die Generierung von Inhalten beeinflusst.[27] Diese bieten jedoch eine gute Grundlage um einem Sprachmodell eine Persönlichkeit für die Beantwortung von Prompts zu geben.[28]

[21] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Generate/generate.ipynb`
[22] `https://platform.openai.com/`
[23] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Embeddings/embeddings.ipynb`
[24] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Similarity/interSimilarity.ipynb`
[25] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Similarity/intraSimilarity.ipynb`
[26] `https://github.com/MaxGit32/data_mining_projekt/blob/main/Generate/generate.ipynb`
[27] Neumann et al. 2025, S. 1.
[28] H. Jiang et al. 2024, S. 3612.

Der Schritt II und III der Implementierungspipeline wurde auf folgender Hardware durchgeführt:

- CPU: AMD Ryzen 7 9700X ×16
- RAM: 32,0 GiB
- Betriebssystem: Ubuntu 24.04.4 LTS
- Python: 3.12.3

Die Verwendung des Embedding-Modells *text-embedding-3-small* verursachte kumuliert über alle Testversuche und Experimentdurchläufe Gesamtkosten von 0,29 Euro bei 14,61 Millionen Input-Tokens.

3.6 Ressourcen für Berechnungen

In der vorliegenden Replikationsarbeit beschränken wir uns auf einen ausgewählten Teil der gewählten Modelle des Originalartikels. Um geeignete Modelle für die zur Verfügung stehende Hardware zu finden, haben wir zunächst den benötigten GPU Speicher für einen Großteil der Modelle kalkuliert. Hierbei haben wir bereits bewusst Modelle ausgeschlossen, die grundsätzlich zu groß für die verfügbare Hardware sind. Für die Replikation wurden 12 Modelle ausgewählt.

Die Autoren der Originalstudie verwendeten für die Durchführung ihrer Experimente NVIDIA A100 sowie H100 GPUs.[29] Um die Replizierbarkeit mit unseren Ressourcen zu prüfen, haben wir eine grobe Bedarfsanalyse durchgeführt. Als Berechnungsgrundlage diente zunächst die Anzahl üblicher durchschnittlicher Output-Token eines DGX Sparks. In unserer Berechnung gingen wir von 12 Modellen aus, welche jeweils 50 Samples für jeden der 100 Prompts beantworten. Somit erhalten wir 60000 Antworten, was bei einer durchschnittlichen Tokenlänge von 250 Tokens einem Gesamtoutput von 15 Mio. Tokens entspricht. Der DGX Spark erreicht im Median einen Output von circa 40 Tokens pro Sekunden[30], wodurch jedes unserer Experimente insgesamt etwa 104 Stunden Durchlaufzeit hätte.

Die tatsächliche Durchlaufzeit des Schritts I (Generierung von Antworten) belief sich auf circa 45 Stunden und 40 Minuten pro Experiment. Insgesamt ergibt sich somit eine für beide experimentelle Durchläufe kumulierte Zeit von 91 Stunden und 20 Minuten. Der Schritt II (Generierung der Embeddings) belief sich auf eine ungefähre Durchlaufzeit von 3 Stunden und 12 Minuten pro experimentellen Durchlauf. Für die Gesamtlaufzeit der Embedding-Generierung ergibt sich somit eine experimentelle kumulierte Durchlaufzeit von 6 Stunden und 24 Minuten. Der Zeitbedarf für den Schritt III ist aufgrund der Geringfügigkeit zu vernachlässigen. Die kumulierte Durchlaufzeit beider Experimente hat circa 97 Stunden und 36 Minuten in Anspruch genommen. Die anfängliche ungefähre Schätzung konnte in der Experimentdurchführung weitestgehend bestätigt werden.

Zur Laufzeit wurde regelmäßig die Ressourcennutzung sowie die Durchlaufzeiten durch die Tools htop und NVIDIA SMI überwacht. Die nachfolgende Abb. 1 zeigt ein Beispiel aus dem Laufzeitmonitoring mit htop. Somit konnten potenzielle Probleme zeitnah identifiziert und behoben werden.

[29] L. Jiang et al. 2025, S. 29.
[30] Bourgoyne 2025.

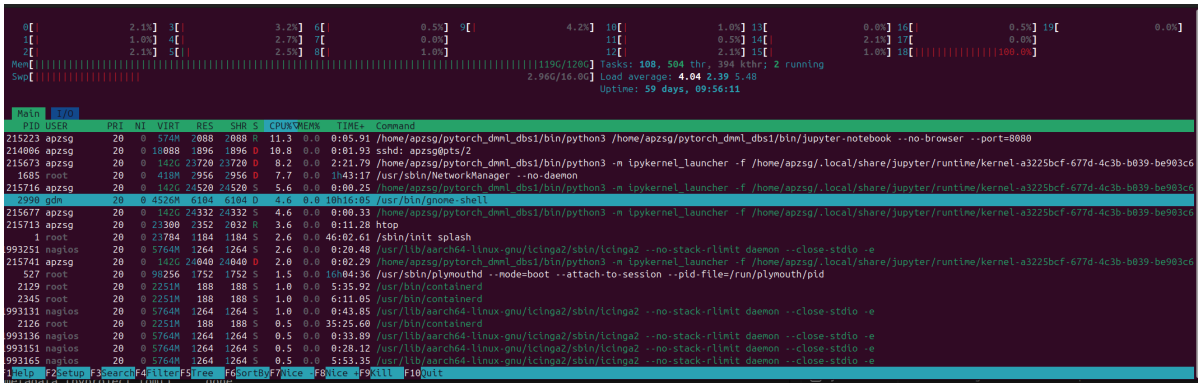

Abbildung 1: Laufzeitmonitoring htop

4 Ergebnisse

In diesem Kapitel werden die gesammelten Ergebnisse der Replikation genauer erläutert und mit der Originalstudie verglichen. Außerdem analysieren wir die Erweiterung der Replikation durch den Einsatz eines Systemprompts. Die thematische Gliederung erfolgt hierfür in Intramodellhomogenität, Intermodellhomogenität und zusätzliche Ergebnisse.

4.1 Replikation

4.1.1 Intramodellhomogenität

In diesem Abschnitt wird die Intramodellhomogenität näher betrachtet. Die Autoren Jiang et al. nutzen zur Auswertung der Originalergebnisse die berechneten Kosinus-Ähnlichkeiten. Für die Darstellung der Replikationsergebnisse wurde sich an der Originalstudie orientiert. Die nachfolgenden Abbildungen stellen somit die Kosinus-Ähnlichkeit, unterteilt in zehn Wertebereiche und die untersuchten Modelle gegenüber. Für jedes Modell ist die durchschnittliche Kosinus-Ähnlichkeit jeder Frage einem Wertebereich zugeordnet und als relative Verteilung in Prozent dargestellt. Die visuelle Darstellung der Ergebnisse der Originalstudie erfolgt als Heatmap. Hierbei beschränkt sich die Darstellung auf die gewählten Modelle der Replikation (Abschnitt 3.1).

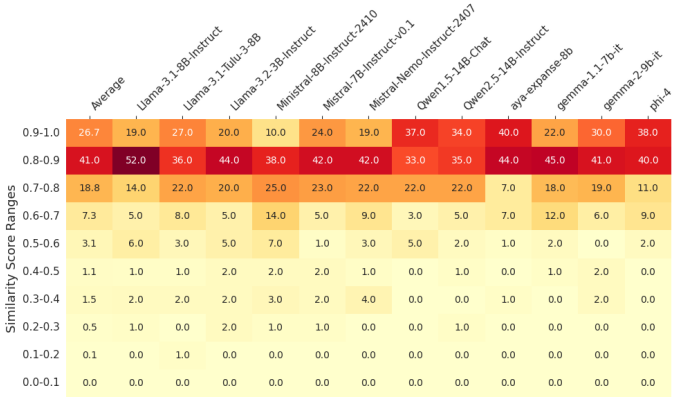

Abbildung 2: Intramodellhomogenitäten (Original)

Die Abb. 2 zeigt die Ergebnisse der Analyse der Intramodellhomogenitäten der in Abschnitt 3.1 gewählten Modelle für die Originalstudie. Die Werte des Originalartikels zeigen für, den zu replizierenden Anteil der LLMs, in 67,7% der Fälle eine Kosinus-Ähnlichkeit über 0,8. Anzumerken ist, dass bei der Gesamtbetrachtung aller Modelle der Originalstudie, inklusive der Closed-Source Modelle, ein Anteil von 79% der Outputs eine Kosinus-Ähnlichkeit von 0,8 überstiegen.[31]

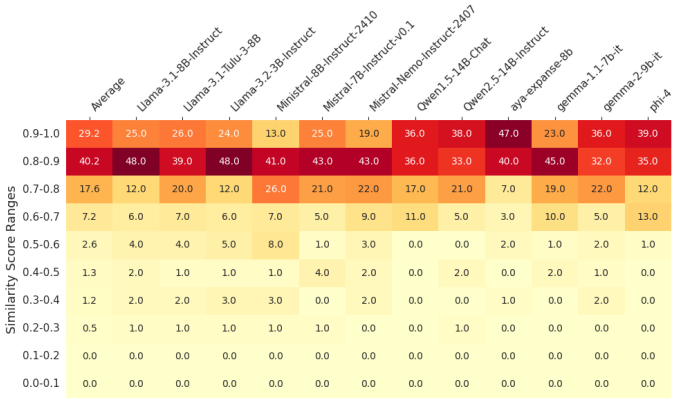

Abbildung 3: Intramodellhomogenitäten (Replikation)

Die Abb. 3 zeigt die Ergebnisse der Analyse der Intramodellhomogenitäten der in Abschnitt 3.1 gewählten Modelle für die Replikation. Für die betrachteten Modelle übersteigt in 69,4% der Fälle die Outputähnlichkeit einen Wert von 0,8. Das obere 25%-Quantil der untersuchten Modelle wird für die Kosinus-Ähnlichkeit von über 0,8 durch die Modelle aya-expanse-8b, phi-4 und Llama-3.1-8B-Instruct abgebildet. Die höchste Outputähnlichkeit weist hierbei das Modell aya-expanse-8b auf. Bei diesem Modell lag 87% der betrachteten Vergleichspaare bei einem Wert über 0,8. Durch die Modelle Llama-3.1-Tulu-3-8B, Ministral-Nemo-Instruct-2407 und Ministral-8B-Instruct-2410 wird das untere 25%-Quantil abgebildet. Die geringste Outputähnlichkeit erreichte in unserer Replikation das Modell Ministral-8B-Instruct-2410, welches lediglich in 54% der untersuchten Fälle eine Ähnlichkeit von 0,8 oder höher aufwies.

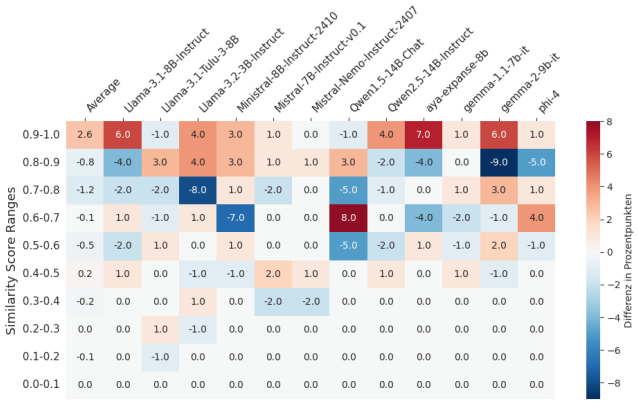

Abbildung 4: Vergleich der Intramodellhomogenitäten

[31] L. Jiang et al. 2025, S. 32.

Bei der Betrachtung der Ergebnisse der Replikation und des Originalartikels zeigt sich grundlegend das gleiche Gesamtbild. Der Großteil der Paar-Ähnlichkeiten liegt im Bereich 0,8-0,9 sowie 0,9-1,0. Dieses Ergebnis bestätigt grundsätzlich die Tendenzen der Intramodellhomogenitäten des Originalartikels.

In der Abb. 4 werden die Ergebnisse der Originalstudie und der Replikation miteinander verglichen. Die abgebildeten Werte der Abb. 4 stellen die Abweichung der Replikationsergebnisse von denen des Originalartikels in Prozentpunkten dar. Im Bereich der Kosinus-Ähnlichkeit über 0,8 sind die Abweichungen von aya-expanse-8b, gemma-1.1-7b-it und Llama-3.1-8B-Instruct auffällig.

Für die Durchschnitte der Bereiche 0,8-0,9 und 0,9-1,0 ergibt sich eine leichte Verschiebung. In der Replikation ist der Anteil an Output im Bereich 0,8-0,9 um 0,8 Prozentpunkte gesunken, im Bereich 0,9-1,0 ist dieser um 2,6 Prozentpunkte angestiegen.

Auffällige Einzelwerte existieren für die Modelle Llama-3.2-3B-Instruct im Bereich 0,7-0,8 mit einer Verringerung um 8 Prozentpunkte, Qwen1.5-14b-Chat im Bereich 0,6-0,7 mit einer Erhöhung um 8 Prozentpunkte, aya-expanse-8b im Bereich 0,9-1,0 mit einer Erhöhung um 7 Prozentpunkte und gemma-2-9b-it im Bereich 0,8-0,9 mit einer Verringerung um 9 Prozentpunkte. Trotz dieser Unterschiede bleibt die Rangfolge der Modelle im Hinblick auf die Verteilungen der Intramodellhomogenitäten weitestgehend unbeeinflusst.

Damit konnten die Ergebnisse bezüglich gewählter Modelle weitestgehend repliziert werden, wobei auffällige Einzelwerte potenziell auf unterschiedliche Implementierungen der Generierungspipeline hindeuten könnten.

4.1.2 Intermodellhomogenität

Dieser Abschnitt legt die Ergebnisse der Untersuchung der Intermodellhomogenität dar. Für die Analyse der Resultate wird in den folgenden Abbildungen ebenfalls die Kosinus-Ähnlichkeit genutzt. Konkret findet sich diese im Schnittpunkt der verglichenen Modelle. Die Grafiken stellen jeweils die betrachteten Modelle an vertikaler und horizontaler Achse gegenüber.

Die Abb. 5 zeigt die Ergebnisse der Analyse der Intermodellhomogenitäten der in Abschnitt 3.1 gewählten Modelle für den Originalartikel. Erkennbar ist eine hohe Ähnlichkeit der generierten Outputs. Die Autoren Jiang et al. merken an, dass sich Inhalte nicht nur kontextuell wiederholen, sondern auch ganze Antwortsätze zwischen verschiedenen Sprachmodellen übereinstimmen. Für die hier konkret gewählten Modelle liegen die Ergebnisse von Jiang et al. zwischen 0,684-0,786. Bei der Betrachtung der Gesamtergebnisse des Originals liegt die Kosinus-Ähnlichkeit zwischen 0,71-0,82.[32]

Die Abb. 6 zeigt die Ergebnisse der Analyse der Intermodellhomogenitäten der in Abschnitt 3.1 gewählten Modelle für die Replikation. Im Durchschnitt liegt die Kosinus-Ähnlichkeit der betrachteten Modelle bei 0,743. Die größte Ähnlichkeit erreicht hierbei das Modell phi-4 mit den Modellen aya-expanse-8b und mit Qwen2.5-14B-Instruct, jeweils mit einem Wert von 0,789. Die geringste Ähnlichkeit besteht zwischen Mistral-7B-Instruct-v0.1 und gemma-2-9b-it, mit einem Wert von 0,695. Die erhaltenen Ähnlichkeitswerte liegen, für die Kreuzungspunkte unterschiedlicher Modelle, zwischen 0,695-0,789 und verzeichnen damit eine eher hohe Outputähnlichkeit.

[32] L. Jiang et al. 2025, S. 5.

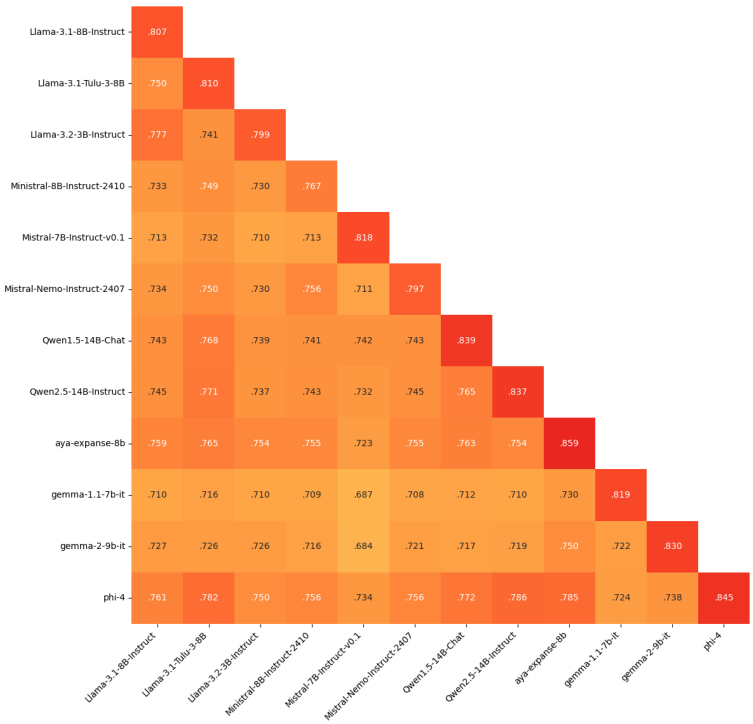

Abbildung 5: Intermodellhomogenitäten (Original)

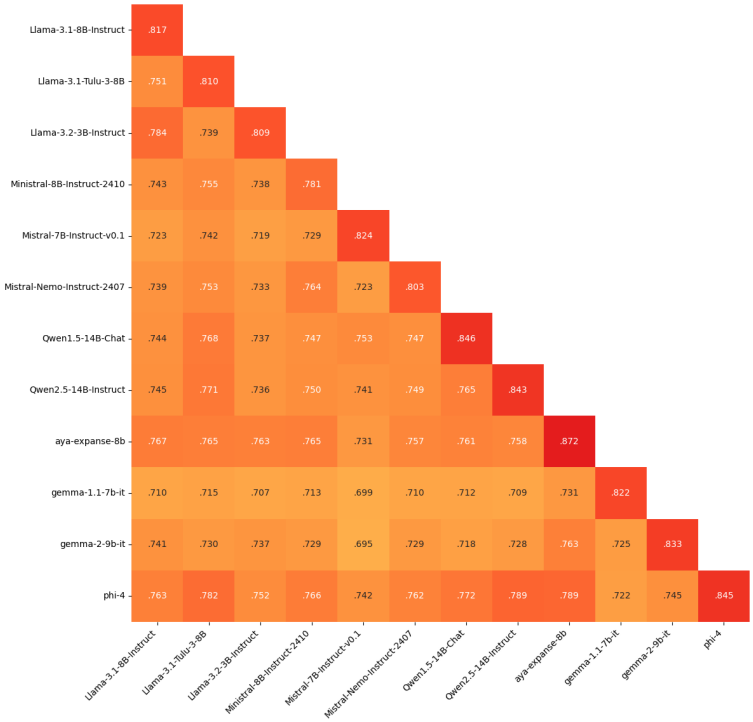

Abbildung 6: Intermodellhomogenitäten (Replikation)

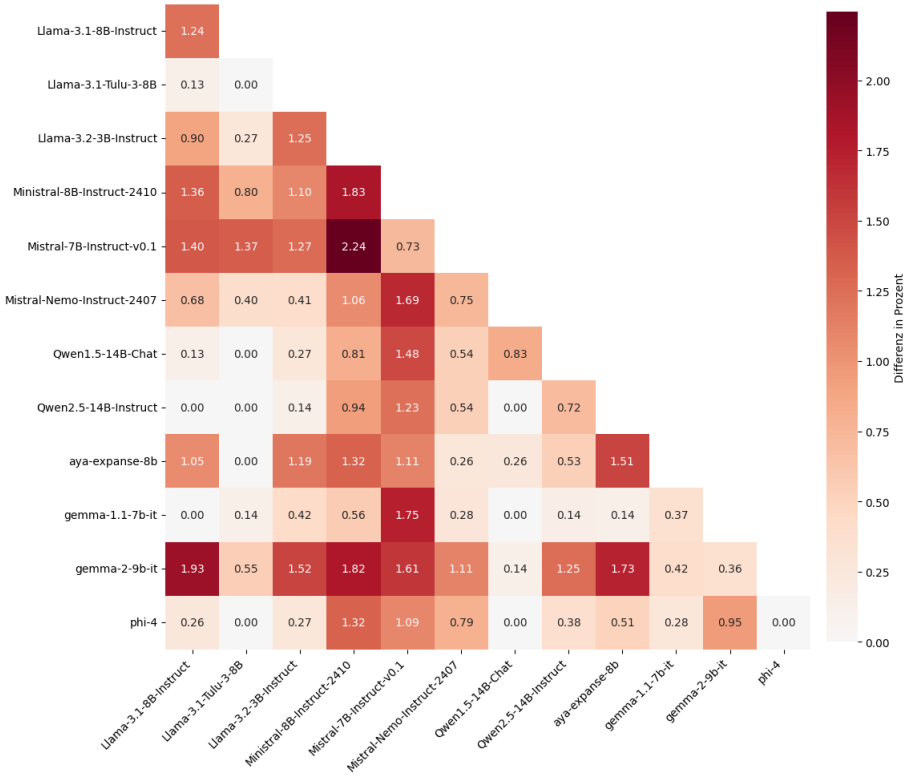

Abbildung 7: Vergleich Intermodellhomogenitäten (Replikation)

Beim Vergleich der Ergebnisse der Replikation mit denen des Originalartikels, weisen diese lediglich eine durchschnittliche Abweichung von 0,74% auf. Die Off-Diagonal-Werte liegen bei Abb. 5 als auch bei Abb. 6 im Bereich 0,68-0,79. Die Diagonal-Werte sind jeweils am höchsten. Eine Ausnahme bildet dabei, äquivalent im Original wie in der Replikation, das Modell Mistral-8B-Instruct-2410 mit einem auffällig geringeren Ähnlichkeitswert als die übrigen Diagonal-Werte. Damit wirkt die Gesamtstruktur der Intermodellhomogenitäten der Replikation mit den Originalergebnissen kongruent.

In den Abb. 5 und 6 bleiben die Größenordnungen der Ähnlichkeitswerte der Modellkombinationen qualitativ ähnlich. So zeigt sich beispielsweise kongruent in beiden Abbildungen, dass das Modell gemma-1.1-7b-it die insgesamt durchschnittlich geringsten Ähnlichkeitswerte eines Modells in allen Paarkombinationen hat.

Die Abb. 7 zeigt die Abweichung der Replikationsergebnisse von den Originalergebnissen in Prozent. Die Abweichungen liegen im Bereich von 0–2,24%. Das zeigt, dass die Originalergebnisse in Bezug auf die Intermodellhomogenitäten gut repliziert werden konnten. Die größte Differenz findet sich hierbei im Vergleich von Mistral-8B-Instruct-v0.1 und Ministral-8B-Instruct-2410 mit 2,24%. Äquivalent dazu zeigt sich, dass die durchschnittlich größten Abweichungen der replizierten Ergebnisse in den Modellkombinationen mit der Mistral-Modellfamilie deutlich werden.

Zusammenfassend zeigt sich, dass die Replikation der Intermodellhomogenitäten erfolgreich war. Die Replikation der Intramodellhomogenitäten zeigt äquivalent zum Originalartikel eine ähnliche Verteilung der Ähnlichkeitswerte in dem Wertebereichen. Trotz dessen zeigt sich eine leichte Abweichung in der Verteilung der Ergebnisse auf die Wertebereiche.

4.2 Zusätzliche Ergebnisse

Bei der Inferenz mit einem LLM können sowohl Systemprompts als auch Userprompts genutzt werden. Das Ziel dieser Erweiterung der Replikation war die Beeinflussung der Intra- sowie Intermodellhomogenitäten durch die Nutzung eines Systemprompts zu untersuchen. Konkret wurde folgender Systemprompt verwendet:

> „Answer the questions with a high degree of ingenuity and creativity, and don't hesitate to draw on unconventional ideas.“

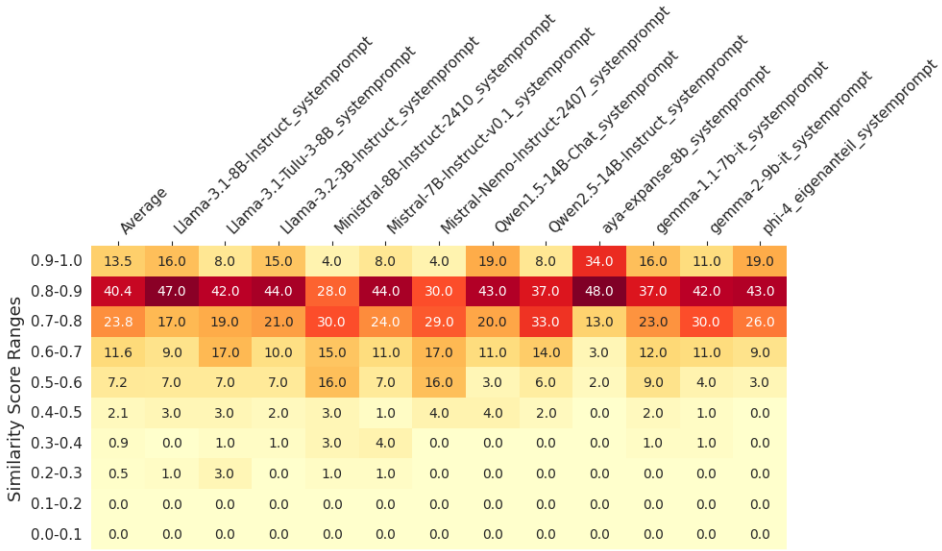

Abbildung 8: Intramodellhomogenitäten (Systemprompt)

Die Abb. 8 zeigt die Ergebnisse der Nutzung des Systemprompts. Der Anteil an generierten Outputs mit einer Kosinus-Ähnlichkeit über 0,8 konnte erheblich verringert werden. Im Durchschnitt liegen nur noch 53,9% des generierten Outputs in den Bereichen über 0,8. Das obere 25%-Quantil der untersuchten Modelle wird für die Kosinus-Ähnlichkeit von über 0,8 ebenfalls durch die Modelle aya-expanse-8b, phi-4 und Llama-3.1-8B-Instruct sowie nun auch Qwen1.5-14B-Instruct abgebildet. Für das untere 25%-Quantil ergibt sich eine leichte Änderung, dieses wird nun durch die Modelle Qwen2.5-14B-Instruct, Mistral-Nemo-Instruct-2407 und Ministral-8B-Instruct-2410 abgebildet. Das Modell Llama-3.1-Tulu-3-8B befindet sich nicht mehr in diesem Bereich.

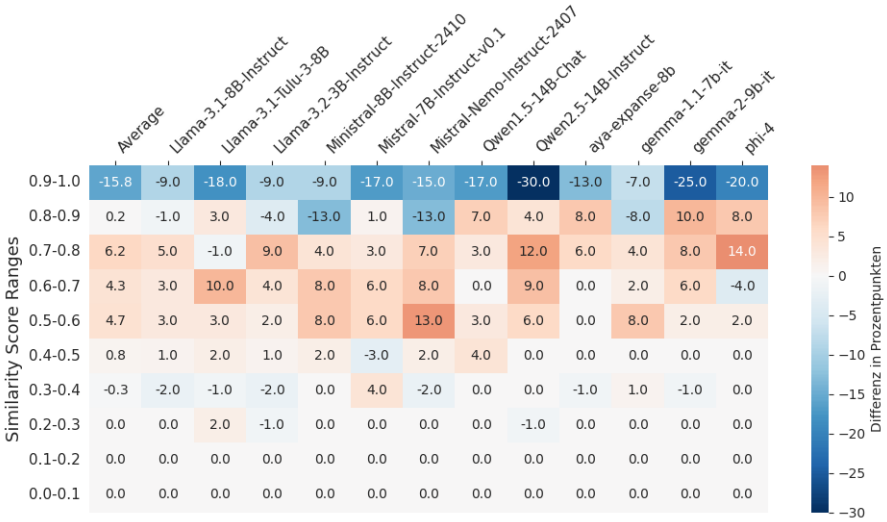

Abbildung 9: Vergleich Intramodellhomogenität (Systemprompt)

Die Abb. 9 zeigt eine durchschnittliche Verringerung des Anteils der Outputähnlichkeit im Bereich von 0,9-1,0 über alle betrachteten Modelle um 15,8 Prozentpunkte. Der Großteil der Paar-Ähnlichkeiten ist hierdurch in den Bereichen 0,7-0,8 sowie 0,8-0,9 verortet. Diese Verschiebung ist ein Hinweis darauf, dass ein Systemprompt die Intramodellhomogenitäten bei LLMs verringern könnte.

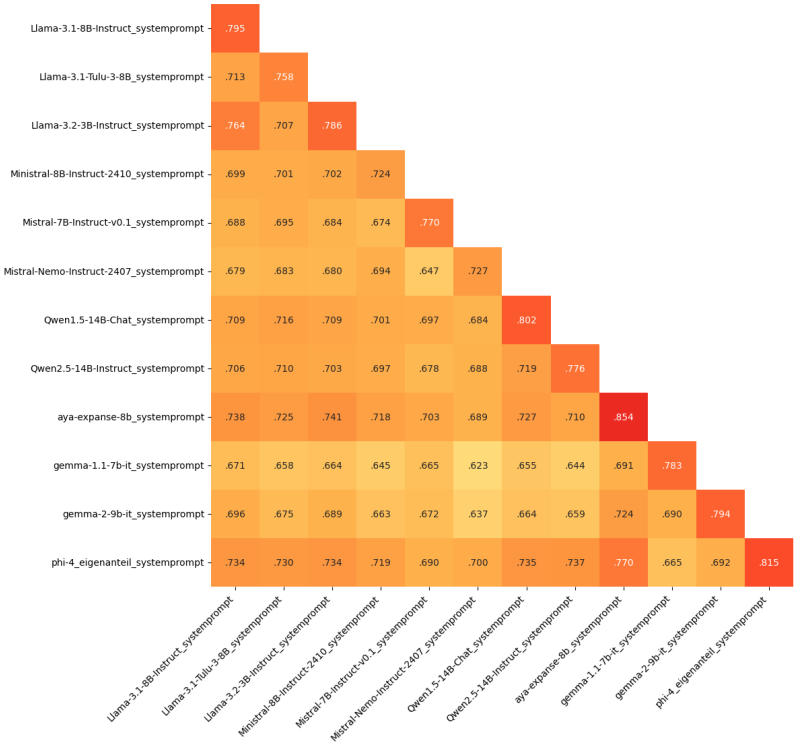

Abbildung 10: Intermodellhomogenitäten (Systemprompt)

Die Abb. 10 stellt die Ähnlichkeiten der erhaltenen Outputs für die Antworten aller Modelle untereinander dar. Die durchschnittliche Kosinus-Ähnlichkeit der betrachteten Modellkombinationen liegt hier bei 0,695. Die größte Ähnlichkeit ist hier für das Modell phi-4 mit dem Modell aya-expanse-8b mit einem Wert von 0,770 zu verzeichnen. Die geringste Ähnlichkeit besteht zwischen den Antworten von Mistral-Nemo-Instruct-2407 und gemma-1.1-7b-it mit einem Wert von 0,623. Der Bereich der erzeugten Werte von 0,623-0,770 kann hier ebenfalls als eher hohe Outputähnlichkeit bezeichnet werden.

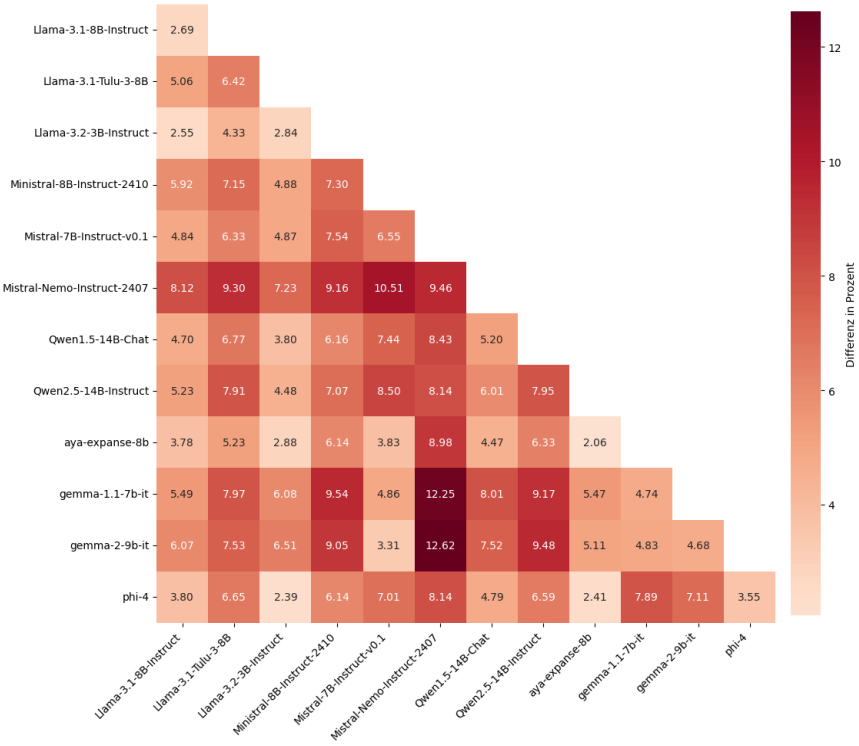

Abbildung 11: Vergleich Intermodellhomogenitäten (Systemprompt)

Die Abb. 11 zeigt die Differenz zwischen Intermodellhomogenitäten der Replikation und denen mit dem Systemprompt erzeugten in Prozent. Es zeigen sich Abweichungen zwischen 2,06-12,62%. Die Intermodellhomogenitäten mit Systemprompt in Abb. 10 haben durchweg niedrigere Ähnlichkeitswerte als die Intermodellhomogenitäten der Replikation in Abb. 6. Im Durchschnitt sind es 6,3% Abweichung nach unten von den ursprünglichen Replikationsergebnissen. Auffällig ist das Modell Mistral-Nemo-Instruct-2407, welches die durchschnittlich höchste Verringerung der Ähnlichkeitswerte über alle Modellkombinationen und die größten Abweichungen nach unten aufweist. Wohingegen zum Beispiel Llama-3.2-3B-Instruct eine deutlich geringere durchschnittliche Verringerung der Ähnlichkeitswerte über alle Modellkombinationen aufweist. Das könnte darauf hinweisen, dass die Verringerung der Intermodellhomogenitäten durch Systemprompts unterschiedlich starke Ausprägungen bei unterschiedlichen Modellen hat. Damit zeigt sich, dass eine signifikante Verringerung der Intermodellhomogenität bei LLMs durch die Verwendung eines Systemprompts möglich sein könnte.

5 Diskussion

Die Intra- und Intermodellhomogenitäten des Originalartikels konnten im Rahmen dieser Replikationsarbeit im Allgemeinen repliziert werden. Abweichungen zwischen der Replikation und dem Originalartikel befanden sich in einem akzeptablen Rahmen, ohne bedeutende qualitative Unterschiede.

Die Erweiterung der Replikationsarbeit um einen Ansatz zur Verringerung der Intra- sowie der Intermodellhomogenitäten zeigte sich erfolgreich. Die Ergebnisse aus Abschnitt 4.2 weisen auf mögliches weiteres Forschungspotenzial hin. Ebenso könnte die Erweiterung der Replikationsarbeit praxisrelevanten Nutzen haben. Als leicht implementierbarer Ansatz des Prompt-Engineerings, könnte ein Systemprompt in der Praxis zu einer Verbesserung der Antwortdiversität von LLMs, auf offene Fragen, führen.

Nachvollziehbar war die Dokumentation der von dem Originalartikel verwendeten Modelle, sodass leicht eine Stichprobe für die Replikation gewählt werden konnte. Des Weiteren war die Implementierung der Berechnung der Kosinus-Ähnlichkeiten und die Implementierung der Heatmaps nachvollziehbar dokumentiert. Im hinterlegten Github Repository des Originalartikels konnte der Code zur Berechnung der Kosinus-Ähnlichkeiten sowie zur Generierung der Heatmaps gefunden und wiederverwendet werden. Zusätzlich sind die Ergebnisse des Originalartikels gut nachvollziehbar dokumentiert.

Eine Herausforderung und die potenziell größte Fehlerquelle stellte die Generierung der Antworten durch die LLMs dar. Aufgrund fehlender Dokumentation der Implementierung der Generierungspipeline und die ausschließliche Angabe der Hyperparameter, muss für die Replikation der Generierungspipeline selbstständig ein geeignetes Tool ausgewählt oder programmiert werden. Diese eigenständige Auswahl kann dazu führen, dass ein Tool oder eine Library genutzt wird, die neben den Hyperparametern Default-Generierungsparameter nutzt, welche abweichend zur Implementierung des Originalartikels sind.

Eine weitere Herausforderung stellte die fehlende Beschreibung des Dateiformats der generierten Embeddings dar. Für das Erstellen der Heatmaps musste der Originalcode angepasst werden, sodass die Embeddings aus dem selbstgewählten Speicherformat geladen werden konnten.

Zur Verbesserung der Replizierbarkeit empfehlen wir Jiang et al. (2025) alle Skripte beziehungsweise Codes sowohl zur Generierung der Antworten als auch zur Generierung der Embeddings bereitzustellen, damit das Speicherformat der Embeddings sowie die Generierung der Antworten durch die LLMs besser nachvollziehbar ist.

Literaturverzeichnis

Bourgoyne, Allen (24. Okt. 2025). *How NVIDIA DGX Spark's Performance Enables Intensive AI Tasks*. NVIDIA. URL: `https://developer.nvidia.com/blog/how-nvidia-dgx-sparks-performance-enables-intensive-ai-tasks/` (besucht am 12. 04. 2026).

Gao, Yunfan et al. (2023). „Retrieval-augmented generation for large language models: A survey". In: *arXiv preprint arXiv:2312.10997* 2.1, S. 32.

Jiang, Hang et al. (2024). „PersonaLLM: Investigating the ability of large language models to express personality traits". In: *Findings of the association for computational linguistics: NAACL 2024*, S. 3605–3627.

Jiang, Liwei et al. (2025). „Artificial hivemind: The open-ended homogeneity of language models (and beyond)". In: *arXiv preprint arXiv:2510.22954*.

Neumann, Anna et al. (2025). „Position is power: System prompts as a mechanism of bias in large language models (llms)". In: *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency*, S. 573–598.

Phung, Tung et al. (2023). „Generative AI for programming education: Benchmarking ChatGPT, GPT-4, and human tutors". In: *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 2*, S. 41–42.

Sordo, Zineb, E Chagnon und D Ushizima (2025). „A Review on Generative AI For Text-To-Image and Image-To-Image Generation and Implications To Scientific Images. arXiv 2025". In: *arXiv preprint cs.CV/2502.21151*.

Zhao, Wenting et al. (2024). „Wildchat: 1m chatgpt interaction logs in the wild". In: *arXiv preprint arXiv:2405.01470*.

Anlagen

Tabelle 2: Liste der betrachteten Modelle des Originalartikels und der gewählten Modelle für die Replikationsarbeit

Typ	Replikation	Original
Open-Source		meta-llama/Llama-3.1-8B-Instruct
		meta-llama/Llama-3.1-70B-Instruct
		meta-llama/Meta-Llama-3.1-405B-Instruct-Turbo
		meta-llama/Llama-3.2-1B-Instruct
	×	meta-llama/Llama-3.2-3B-Instruct
		meta-llama/Llama-3.3-70B-Instruct
		google/gemma-2-2b-it
	×	google/gemma-2-9b-it
		google/gemma-2-27b-it
		google/gemma-1.1-2b-it
	×	google/gemma-1.1-7b-it
		Qwen/Qwen1.5-0.5B-Chat
		Qwen/Qwen1.5-1.8B-Chat
		Qwen/Qwen1.5-4B-Chat
		Qwen/Qwen1.5-7B-Chat
	×	Qwen/Qwen1.5-14B-Chat
		Qwen/Qwen1.5-32B-Chat
		Qwen/Qwen1.5-72B-Chat
		Qwen/Qwen1.5-110B-Chat
		Qwen/Qwen2-0.5B-Instruct
		Qwen/Qwen2-1.5B-Instruct
		Qwen/Qwen2-72B-Instruct
		Qwen/Qwen2.5-0.5B-Instruct
		Qwen/Qwen2.5-1.5B-Instruct
		Qwen/Qwen2.5-3B-Instruct
		Qwen/Qwen2.5-7B-Instruct
	×	Qwen/Qwen2.5-14B-Instruct
		Qwen/Qwen2.5-32B-Instruct
		Qwen/Qwen2.5-72B-Instruct
		Qwen/Qwen2.5-7B-Instruct-1M
		Qwen/Qwen2.5-14B-Instruct-1M
		Qwen/Qwen3-0.6B
		Qwen/Qwen3-1.7B
		Qwen/Qwen3-4B
		Qwen/Qwen3-8B
		Qwen/Qwen3-14B
		Qwen/Qwen3-32B
		deepseek-ai/DeepSeek-V3
		mistralai/Mistral-Small-24B-Instruct-2501
	×	mistralai/Mistral-7B-Instruct-v0.1
		mistralai/Mistral-7B-Instruct-v0.2
		mistralai/Mistral-7B-Instruct-v0.3
	×	mistralai/Ministral-8B-Instruct-2410
	×	mistralai/Mistral-Nemo-Instruct-2407
		mistralai/Mistral-Small-Instruct-2409
		mistralai/Mistral-Large-Instruct-2411
		mistralai/Mixtral-8x7B-Instruct-v0.1
		microsoft/Phi-3-mini-128k-instruct
		microsoft/Phi-3.5-mini-instruct
	×	microsoft/phi-4
	×	CohereForAI/aya-expanse-8b
		CohereForAI/aya-expanse-32b
		CohereForAI/c4ai-command-r-plus-08-2024
		CohereForAI/c4ai-command-r-08-2024
		allenai/OLMo-2-1124-13B-Instruct
		allenai/OLMo-2-1124-7B-Instruct
	×	allenai/Llama-3.1-Tulu-3-8B
		allenai/Llama-3.1-Tulu-3-70B
Closed-Source		gpt-4o-2024-11-20
		gpt-4o-2024-08-06
		gpt-4o-2024-05-13
		gpt-4o-mini-2024-07-18
		gpt-4-turbo-2024-04-09
		claude-3-5-sonnet-20241022
		claude-3-5-haiku-20241022
		claude-3-sonnet-20240229
		claude-3-haiku-20240307
		claude-3-opus-20240229
		gemini-1.5-flash
		gemini-1.5-pro
		gemini-2.0-flash
		gemini-2.0-flash-lite-preview-02-05
		qwen-max-2025-01-25
		qwen-plus-2025-01-25
		qwen-turbo-2024-11-01